

Talk with Your Data

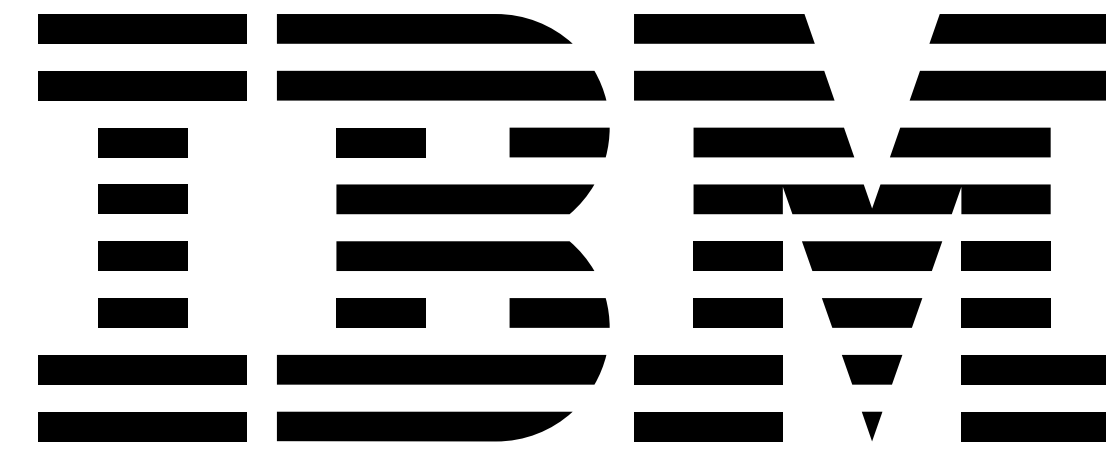
Natural Language Data Access

Hibernate - Quarkus - LangChain4j

About Me

Marco Belladelli

- Software Developer @ IBM
- Hibernate team, ORM maintainer
- Quarkus and LangChain4j contributor
- <https://github.com/mbellade>
- marco@hibernate.org



Relational Databases

Tabular Data



Owner	Country	File_Date	IPC_Class
Company A	US	6/18/2008	H05H13
Company A	EP	1/30/1998	A61N5
Company A	EP	1/30/1998	A61N5
Company A	EP	1/30/1998	A61N5
Company A	JP	8/28/1997	A61N5
Company A	JP	10/4/2002	A61N5
Company A	JP	1/27/2003	A61N5
Company A	JP	4/14/2003	A61N5
Company A	JP	5/13/2011	A61N5
Company B	JP	4/2/1998	G12B13
Company B	JP	4/2/1998	G12B13
Company B	JP	5/28/1997	A61N5
Company B	JP	11/12/1997	A61N5
Company B	JP	2/29/2000	A61N5
Company B	JP	4/30/2002	A61N5

Java Applications

Classes & Objects

```
@Entity  👤 Marco Belladelli *
@Table(name = "company_table")
public class Company implements Serializable {
    @Id
    private long id;

    @Column(nullable = false)  3 usages
    private String name;

    @Embedded  3 usages
    private Address address;

    @OneToMany(mappedBy = "company")  3 usages
    private List<Employee> employees;

    public Company() {  👤 Marco Belladelli
    }

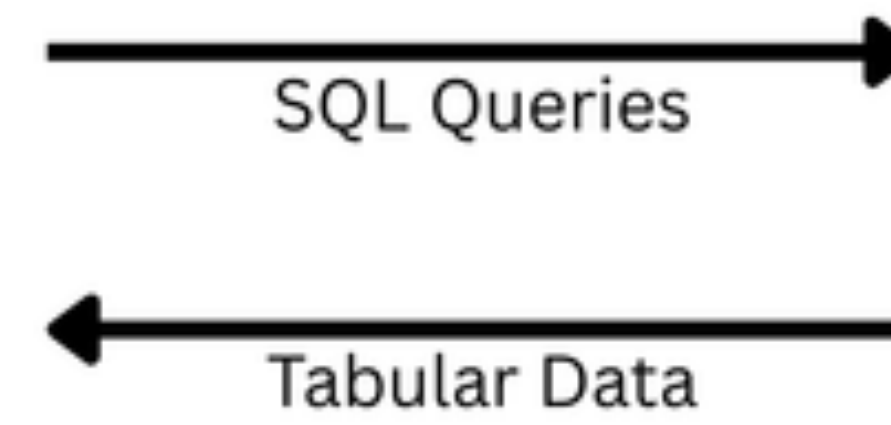
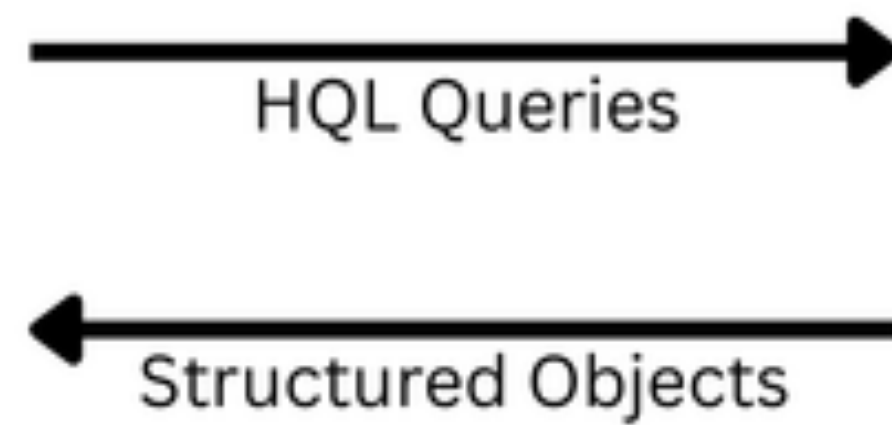
    public Company(long id, String name, Address address) {  1 usage
        this.id = id;
        this.name = name;
        this.address = address;
        this.employees = new ArrayList<>();
    }

    public long getId() { return id; }
```



Hibernate ORM

Solution to the object/relational impedance mismatch



The OSS Java Data Intelligence Stack



The bridge between
Java Objects and
Relational Databases



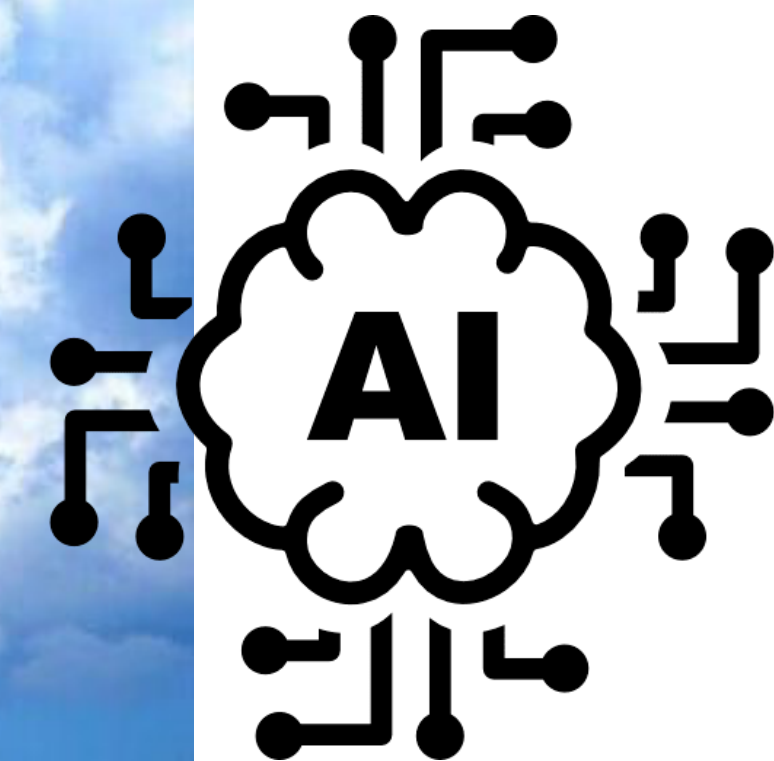
Java Runtime with fast
startup, live reload,
and a rich extension
ecosystem.



The emerging
framework for LLM
integration in Java -
unified API for AI
model orchestration.

LangChain4j

The “Hibernate” of AI



AI Models

```
@Entity  ⚡ Marco Belladelli *
@Table(name = "company_table")
public class Company implements Serializable {
    @Id
    private long id;

    @Column(nullable = false)  3 usages
    private String name;

    @Embedded  3 usages
    private Address address;

    @OneToMany(mappedBy = "company")  3 usages
    private List<Employee> employees;

    public Company() {  ⚡ Marco Belladelli
    }

    public Company(long id, String name, Address
        this.id = id;
        this.name = name;
        this.address = address;
        this.employees = new ArrayList<>();
    }

    public long getId() { return id; }
```



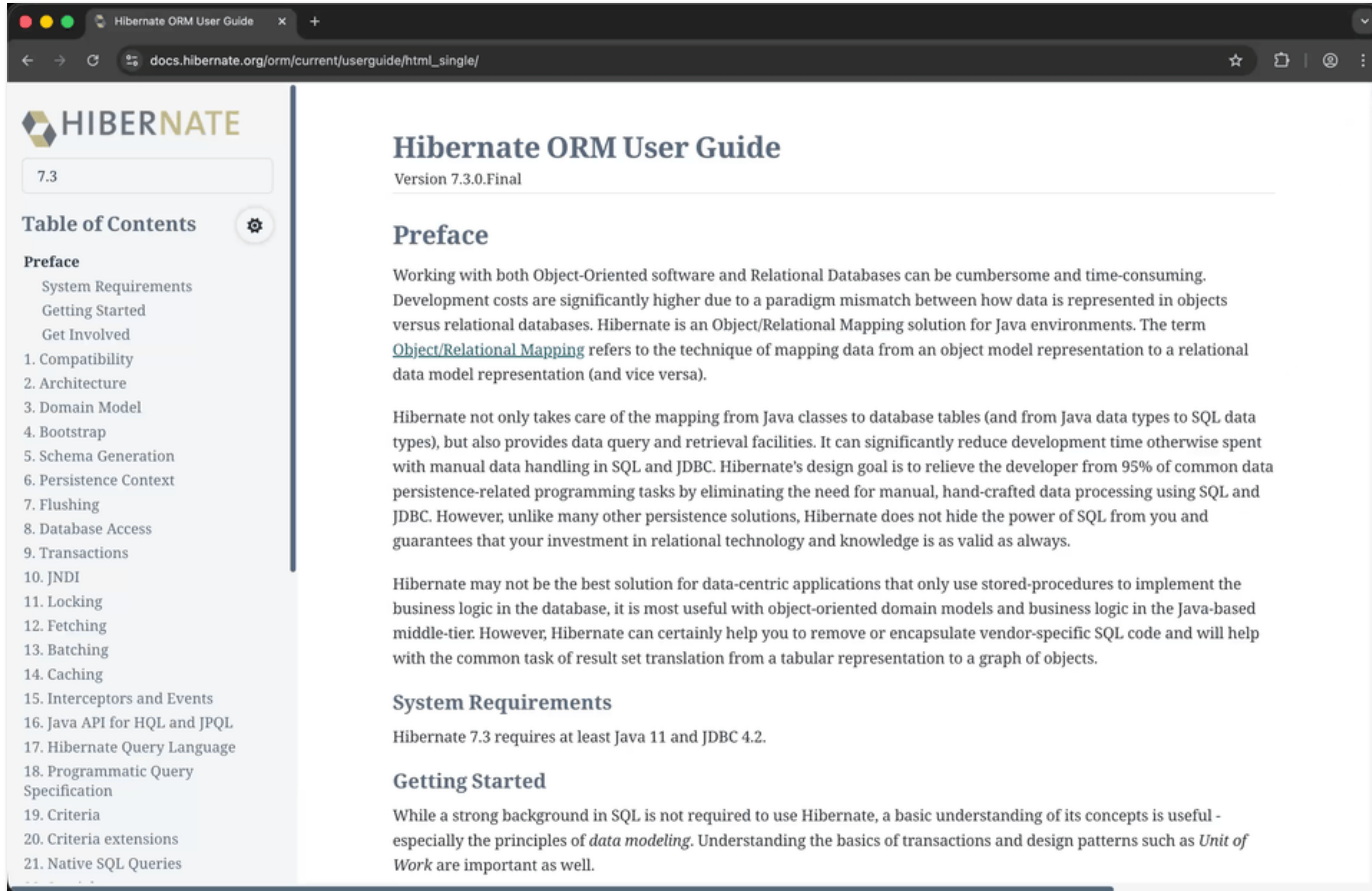
Owner	Country	File_Date	IPC_Class
Company A	US	6/18/2008	H05H13
		1/30/1998	A61N5
		1/30/1998	A61N5
		1/30/1998	A61N5
		8/28/1997	A61N5
		10/4/2002	A61N5
		1/27/2003	A61N5
		4/14/2003	A61N5
		5/13/2011	A61N5
		4/2/1998	G12B13
		4/2/1998	G12B13
		5/28/1997	A61N5
		11/12/1997	A61N5
		2/29/2000	A61N5
		4/30/2002	A61N5

AI Models



AI Models

Trained on vast amounts of public data

A screenshot of a web browser displaying the Hibernate ORM User Guide. The browser's address bar shows the URL 'docs.hibernate.org/orm/current/userguide/html_single/'. The page features a sidebar on the left with the Hibernate logo and a 'Table of Contents' section listing various topics from 'Preface' to 'Native SQL Queries'. The main content area is titled 'Hibernate ORM User Guide' with the subtitle 'Version 7.3.0.Final'. It includes a 'Preface' section explaining the purpose of Hibernate, a 'System Requirements' section stating the need for Java 11 and JDBC 4.2, and a 'Getting Started' section. The page is styled with a clean, professional layout using a sans-serif font.

HIBERNATE

7.3

Table of Contents

- Preface
 - System Requirements
 - Getting Started
 - Get Involved
- 1. Compatibility
- 2. Architecture
- 3. Domain Model
- 4. Bootstrap
- 5. Schema Generation
- 6. Persistence Context
- 7. Flushing
- 8. Database Access
- 9. Transactions
- 10. JNDI
- 11. Locking
- 12. Fetching
- 13. Batching
- 14. Caching
- 15. Interceptors and Events
- 16. Java API for HQL and JPQL
- 17. Hibernate Query Language
- 18. Programmatic Query Specification
- 19. Criteria
- 20. Criteria extensions
- 21. Native SQL Queries

Hibernate ORM User Guide

Version 7.3.0.Final

Preface

Working with both Object-Oriented software and Relational Databases can be cumbersome and time-consuming. Development costs are significantly higher due to a paradigm mismatch between how data is represented in objects versus relational databases. Hibernate is an Object/Relational Mapping solution for Java environments. The term [Object/Relational Mapping](#) refers to the technique of mapping data from an object model representation to a relational data model representation (and vice versa).

Hibernate not only takes care of the mapping from Java classes to database tables (and from Java data types to SQL data types), but also provides data query and retrieval facilities. It can significantly reduce development time otherwise spent with manual data handling in SQL and JDBC. Hibernate's design goal is to relieve the developer from 95% of common data persistence-related programming tasks by eliminating the need for manual, hand-crafted data processing using SQL and JDBC. However, unlike many other persistence solutions, Hibernate does not hide the power of SQL from you and guarantees that your investment in relational technology and knowledge is as valid as always.

Hibernate may not be the best solution for data-centric applications that only use stored-procedures to implement the business logic in the database, it is most useful with object-oriented domain models and business logic in the Java-based middle-tier. However, Hibernate can certainly help you to remove or encapsulate vendor-specific SQL code and will help with the common task of result set translation from a tabular representation to a graph of objects.

System Requirements

Hibernate 7.3 requires at least Java 11 and JDBC 4.2.

Getting Started

While a strong background in SQL is not required to use Hibernate, a basic understanding of its concepts is useful - especially the principles of *data modeling*. Understanding the basics of transactions and design patterns such as *Unit of Work* are important as well.

INTRODUCING

hibernate-assistant

New module that acts as a bridge between your mapped domain model and LLMs.

1. Metamodel Structure

Describes your entity model, i.e. your entities, their attributes, and the associations between them in a textual format consumable by LLMs.

2. Query Results Serialization

Transforms complex ORM query results: objects, collections, embeddables, associations for AI agents to consume.

Building the assistant

Live Demo

Extract all companies with a name starting with A

Id	Address	Employees	Name								
5	- <i>Object{3 properties}</i>	<uninitialized>	Alphabet								
	<table><tr><th>Property</th><th>Value</th></tr><tr><td>city</td><td>Mountain View</td></tr><tr><td>country</td><td>USA</td></tr><tr><td>street</td><td>Googleplex</td></tr></table>			Property	Value	city	Mountain View	country	USA	street	Googleplex
	Property			Value							
	city			Mountain View							
	country			USA							
street	Googleplex										
6	+ <i>Object{3 properties}</i>	<uninitialized>	American Express								
7	+ <i>Object{3 properties}</i>	<uninitialized>	Apple								

```
SELECT c FROM Company c WHERE LOWER(c.name)
LIKE 'a%'
```


AI • CODING

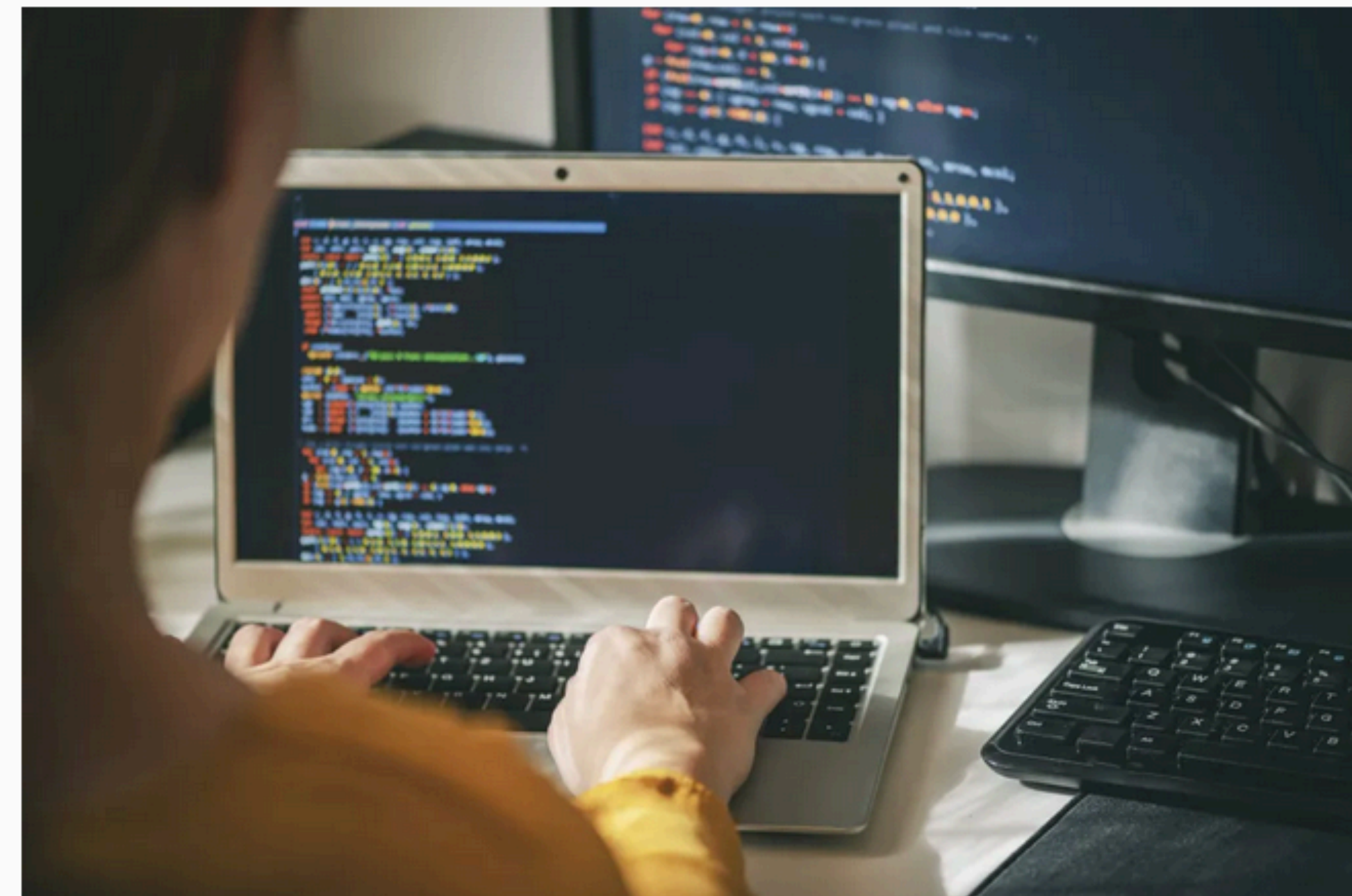
An AI-powered coding tool wiped out a software company's database, then apologized for a 'catastrophic failure on my part'

By Beatrice Nolan

Tech Reporter

July 23, 2025, 7:22 AM ET

[Add us on](#)  



AI has significant potential to accelerate software development, with most Big Tech companies already leaning on AI tools for internal coding capacity.

GETTY IMAGES

Why Hibernate?

Why should we trust you?

- **Constrained Access:** LLMs can only access mapped entities and their fields - no arbitrary table access, and can be restricted to read-only queries
- **Fail-Early Validation:** Invalid HQL is caught before it reaches the database
- **Self-Correction:** Hibernate's clear error messages can be fed back to the LLM to fix mistakes, agentic patterns will produce better results over time
- **Portability:** One query language (HQL) works across all supported databases
- **Handling Complexity:** HQL is closer to our language — associations, embeddables, inheritance are simpler, and complex query logic becomes natural path expressions

RAG

Retrieval-augmented Generation

- Enrich the LLM's responses with **real information** from your database
- Retrieve **relevant data** first, then let the LLM reason about it
- The LLM can now provide **informed, contextual answers** grounded in your actual data



Implementing RAG

Live Demo



Hibernate Assistant

Hello! I'm your Hibernate Assistant. Ask me anything about data.

How many USA companies with at least 2 employees do we have on record?

There are 4 USA companies with at least 2 employees on record.

```
SELECT COUNT(c) FROM Company c
WHERE c.address.country = 'USA'
AND SIZE(c.employees) >= 2
```


LangChain4j integrations

langchain4j-hibernate, langchain4j-experimental-hibernate

This screenshot shows a GitHub pull request for the 'langchain4j' repository. The title is 'Add HibernateEmbeddingStore integration #4622'. It was merged by 'dliubarskyi' 5 commits ago into the 'langchain4j:main' branch from the 'beikov:hibernate-integration' branch. The pull request has 34 conversations, 5 commits, 7 checks, and 37 files changed. The 'Issue' section is empty. The 'Change' section describes adding a new 'EmbeddingStore' implementation based on Hibernate ORM and its hibernate-vector module. It lists several benefits: making existing SQL and NoSQL integrations obsolete, allowing more vector distance operations, and enabling HQL queries. A note at the bottom states that this integration will free developers from maintaining multiple separate 'EmbeddingStore' implementations.

langchain4j / langchain4j

Code Issues 575 Pull requests 132 Agents Discussions Actions Projects Security Insights

Add HibernateEmbeddingStore integration #4622

Merged dliubarskyi merged 5 commits into langchain4j:main from beikov:hibernate-integration last month

Conversation 34 Commits 5 Checks 7 Files changed 37

beikov commented last month · edited Contributor

Issue

None yet

Change

Add a new `EmbeddingStore` implementation based on Hibernate ORM and its hibernate-vector module.

- Could make existing SQL specific Langchain4j `EmbeddingStore` integrations obsolete
- Makes more SQL vector stores available due to Hibernate ORM supporting them
- Could make existing NoSQL Langchain4j `EmbeddingStore` integrations obsolete when we get further with our Hibernate NoSQL efforts
- Allows more vector distance operations and vector types thanks to Hibernate ORM supporting them
- Allows more convenient data model customizations by defining fields on entity classes
- Allows evolving existing entity classes into a source for `EmbeddingStore` API
- Another benefit is that when users use entities, they can then also use HQL to write custom and more complex vector/hybrid-search queries if needed

Thanks to the abstraction, this integration would free Langchain4j developers from maintainance of all those separate `EmbeddingStore` implementations and focus on just a single integration, with our help.

This screenshot shows a GitHub pull request for the 'langchain4j' repository. The title is 'Add HibernateContentRetriever for data retrieval through HQL queries'. It was merged by 'dliubarskyi' 1 commit ago into the 'langchain4j:main' branch from the 'mbellade:hibernate-content-retriever' branch. The pull request has 11 conversations, 1 commit, 7 checks, and 11 files changed. The 'Issue' section is empty. The 'Change' section describes adding a new 'HibernateContentRetriever' similar to the existing SQL-based one, but taking advantage of 'Hibernate ORM' and its new 'hibernate-assistant' module. It lists features like context initialization prompt, constrained access, HQL support, and query results serialization. A 'General checklist' at the bottom shows three items are checked: no breaking changes, unit/integration tests added, and tests covering both positive and negative cases.

langchain4j / langchain4j

Code Issues 575 Pull requests 132 Agents Discussions Actions Projects Security Insights

Add HibernateContentRetriever for data retrieval through HQL queries

Merged dliubarskyi merged 1 commit into langchain4j:main from mbellade:hibernate-content-retriever last week

Conversation 11 Commits 1 Checks 7 Files changed 11

mbellade commented last week · edited Contributor

Issue

None yet

Change

Add new `HibernateContentRetriever`, similar to the existing SQL-based one, but taking advantage of [Hibernate ORM](#) and its new `hibernate-assistant` module to provide:

- context initialization prompt based on the runtime metamodel (mapped entity classes and corresponding db structures)
- constrained access only to the mapped tables and columns
- HQL (Hibernate Query Language) support, much closer to natural language with advanced functionality
- `SELECT` -only queries, guaranteed by Hibernate's query parsing
- query results serialization, with handling of complex data types, lazy properties and circular associations

General checklist

- ✓ There are no breaking changes (API, behaviour)
- ✓ I have added unit and/or integration tests for my change
- ✓ The tests cover both positive and negative cases

Context Matters

Live demo

 **Hibernate Assistant**

Extract all employees of the "Red Hat" company

Id	Company	Firstname	Lastname	Projects	Salary										
1	- <i>Object{4 properties}</i>	Marco	Belladelli	<uninitialized>	500000										
	<table><thead><tr><th>Property</th><th>Value</th></tr></thead><tbody><tr><td>id</td><td>1</td></tr><tr><td>address</td><td>+ <i>Object{3 properties}</i></td></tr><tr><td>employees</td><td><uninitialized></td></tr><tr><td>name</td><td>Red Hat</td></tr></tbody></table>	Property	Value	id	1	address	+ <i>Object{3 properties}</i>	employees	<uninitialized>	name	Red Hat				
Property	Value														
id	1														
address	+ <i>Object{3 properties}</i>														
employees	<uninitialized>														
name	Red Hat														
2	+ <i>Object{4 properties}</i>	Andrea	Boriero	<uninitialized>	100000										

Which projects are these employees working on?

These employees are working on the Hibernate and Quarkus projects.

Are there any employees from other companies working on these projects?

Yes, there is an employee from IBM working on these projects.

What's their name?

Their name is Luca Molteni.

MCP

Model Context Protocol

- Open standard for **connecting** AI models with tools and data providers
- Single API integration, acts like a **universal adapter** for AI agents
- Your Hibernate Assistant becomes a fundamental **data tool** that any agent can discover and use



MCP

Live Demo

`npx @modelcontextprotocol/inspector`

MCP Inspector v0.21.1

Transport Type

Streamable HTTP

URL

http://localhost:8080/mcp

Connection Type

Via Proxy

Server Entry

Servers File

> Authentication

> Configuration

Reconnect

Disconnect

● Connected

Resources

Prompts

Tools

Tasks

Apps

Ping

Sampling

Elicitations

Roots

Auth

Metadata

Tools

List Tools

Clear

hibernate_query

Execute a query against the database using the Hibernate Assistant.
Pass a natural language message and get a natural language response...

hibernate_query_json

Execute a query against the database using the Hibernate Assistant.
Pass a natural language message and get a JSON representation of th...

hibernate_get_metamodel

Retrieve a textual (JSON) representation of the Hibernate Metamodel,
i.e. the entities, properties and relationships defined in the persistence...

hibernate_query

Execute a query against the database using the Hibernate Assistant. Pass a natural language message and get a natural language response based on the results of the query.

× Read-only

✓ Destructive

× Idempotent

✓ Open-world

query *

Natural language query to execute

Tool-specific Metadata:

No metadata pairs.

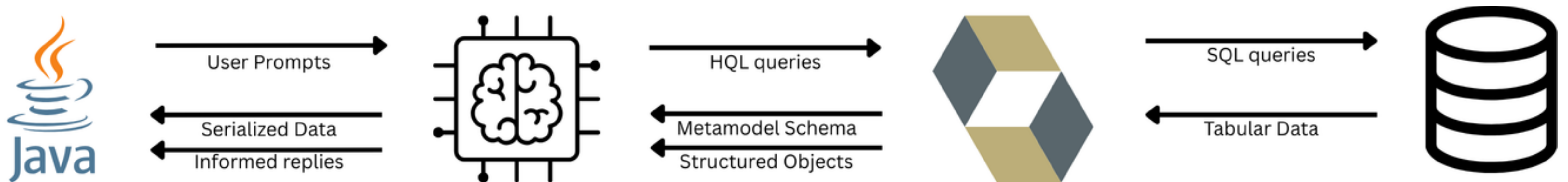
Add Pair

Run Tool

Copy Input

The full picture

Agentic database interaction, powered by Hibernate



Takeaways

1. Hibernate already **knows your domain**: the Assistant module simply exposes that knowledge to LLMs
2. **HQL over SQL**: type safety, constrained access, fail-early validation, better portability
3. Advanced queries made **simple**: anyone can use Hibernate through natural language
4. RAG with **real data**: ground LLM responses in your actual database
5. MCP makes it **extensible**: expose your data to any AI agent in the ecosystem
6. **Minimal code**: a few classes, a few annotations, and your data speaks natural language

Getting started

Try it yourself!

- New hibernate-assistant module - available since ORM 7.3+
- Quarkus LangChain4j integration: `quarkus-langchain4j`-*
- Any supported LLM provider (Ollama, OpenAI, Anthropic, Google, ...)
- Optionals
 - Quarkus MCP servers: `quarkus-mcp-server`-*
 - Quarkiverse Chappie extension: `quarkus-chappie`



Getting started

Resources

- Hibernate ORM: <https://hibernate.org/orm/>
- Quarkus: <https://quarkus.io/>
- LangChain4j: <https://github.com/langchain4j/langchain4j>
- Demo source code: <https://github.com/mbellade/demos>



Demos

Thank you!

Questions?

